

Five Guys - Relationship File Manager

MongoDB CRUD Queries

Project Info

Team: Five Guys

Project: Relationship File Manager

Database: MongoDB (NoSQL)

Backend: Node.js

Collections: users, documents, nodes

1. Collection Structure (Schema Design)

```
// users collection
{
  username: String (unique),
  email: String (unique),
  password_hash: String,
  display_name: String,
  created_at: Date
}

// documents collection (with embedded nodes, relationships, attributes)
{
  user_id: ObjectId (ref: users),
  title: String,
  content: String,
  is_public: Boolean,
  nodes: [
    {
      label: String,
      node_type: String,
      position: { x: Float, y: Float, z: Float },
      attributes: { key: value, ... }
    }
  ],
  relationships: [
    {
      source: String,
      target: String,
      type: String,
      description: String
    }
  ],
  created_at: Date,
  updated_at: Date
}

// nodes collection (for 3D graph queries)
{
  user_id: ObjectId (ref: users),
  doc_id: ObjectId (ref: documents),
  label: String,
  node_type: String,
```

```
position: { x: Float, y: Float, z: Float }
}
```

2. INSERT (Create)

```
// Insert users
db.users.insertMany([
  { username: "ocean1229", email: "kangdh@example.com",
    display_name: "Kang Doohyen", created_at: new Date() },
  { username: "victor_kim", email: "yushin@example.com",
    display_name: "Kim Yushin", created_at: new Date() },
  { username: "guingguing", email: "jinseo@example.com",
    display_name: "Kim Jinseo", created_at: new Date() }
])

// Insert document with embedded nodes and relationships
db.documents.insertOne({
  user_id: userIds[0],
  title: "Software Architecture",
  content: "# Software Architecture

@DesignPattern relates to @SOLID",
  is_public: true,
  nodes: [
    { label: "DesignPattern", node_type: "concept",
      position: { x: 0.0, y: 1.0, z: 0.5 },
      attributes: { type: "technical", priority: "high" } },
    { label: "SOLID", node_type: "concept",
      position: { x: 2.0, y: 1.0, z: 0.0 },
      attributes: { type: "technical", principles: "SRP, OCP, LSP, ISP, DIP" } }
  ],
  relationships: [
    { source: "DesignPattern", target: "SOLID",
      type: "relates_to",
      description: "Design patterns implement SOLID principles" }
  ],
  created_at: new Date(), updated_at: new Date()
})

// Insert nodes for 3D graph
db.nodes.insertMany([
  { user_id: userIds[0], doc_id: docIds[0],
    label: "DesignPattern", node_type: "concept",
    position: { x: 0, y: 1, z: 0.5 } },
  { user_id: userIds[0], doc_id: docIds[0],
    label: "SOLID", node_type: "concept",
    position: { x: 2, y: 1, z: 0 } }
])
```

3. FIND (Read / Select)

```
// Find all users
db.users.find({})

// Find user documents
```

```

db.documents.find({ user_id: userIds[0] })

// Find public documents
db.documents.find({ is_public: true })

// Find documents containing technology nodes ( )
db.documents.find({
  nodes: { : { node_type: "technology" } }
})

// Aggregation: documents per user with total node count
db.documents.aggregate([
  { : {
    _id: "",
    doc_count: { : 1 },
    total_nodes: { : { : "" } }
  }}
])

// Find documents with specific relationship type
db.documents.find({ "relationships.type": "contains" })

// Find node attributes
db.documents.findOne(
  { title: "Software Architecture" },
  { "nodes.label": 1, "nodes.attributes": 1 }
)

```

4. UPDATE

```

// Update document title
db.documents.updateOne(
  { _id: docId },
  { : { title: "Software Architecture Patterns", updated_at: new Date() } }
)

// Update embedded node position
db.documents.updateOne(
  { _id: docId, "nodes.label": "DesignPattern" },
  { : { "nodes.$.position": { x: 2.5, y: 3.0, z: 1.5 } } }
)

// Update node attribute
db.documents.updateOne(
  { _id: docId, "nodes.label": "DesignPattern" },
  { : { "nodes.$.attributes.priority": "critical" } }
)

// Add new attribute to node
db.documents.updateOne(
  { _id: docId, "nodes.label": "SOLID" },
  { : { "nodes.$.attributes.importance": "fundamental" } }
)

// Update user display name
db.users.updateOne(
  { username: "ocean1229" },
  { : { display_name: "DH Kang" } }
)

```

5. DELETE (Remove)

```
// Delete attribute from embedded node ()
db.documents.updateOne(
  { _id: docId, "nodes.label": "KangDH" },
  { : { "nodes.$.attributes.status": "" } }
)

// Delete relationship from document ()
db.documents.updateOne(
  { _id: docId },
  { : { relationships: { source: "Software Architecture", target: "SOLID" } } }
)

// Delete a document
db.documents.deleteOne({ _id: docId })

// Delete a user
db.users.deleteOne({ username: "guingguing" })
```